Stingray Technologies

Bearing Degradation Analysis and Remaining Useful Life (RUL) Prediction

June 12, 2025

Contents

1 Introduction **3**

2 Dataset Overview **3**

3 Methodology **3**

3.1 Data Loading and Preprocessing 3

3.2 Feature Extraction: RRMS and IRRMS 4

3.3 IRRMS Estimation with Outlier Detection and Trend Filtering 4

3.4 Degradation Stage Label Assignment 6

3.5 6. Model Training: Support Vector Machine (SVM) 7

4 Life Progression Estimation **9**

4.1 Overview 9

4.2 Local Slope Estimation 10

4.3 Hybrid Degradation Logic 10

4.4 Code Support (Excerpt) 11

5 Remaining Useful Life (RUL) Prediction **11**

5.1 Overview 11

5.2 Mathematical Formula 11

5.3 Interpretation 11

5.4 Code Support (Excerpt) 12

6 Fault Condition Classification **12**

6.1 Post-RUL Thresholding for Decision Making 12

6.2 Logic Description 12

6.3 Code Snippet (Condition Classification) 12

7 Conclusion **13**

1. Introduction

Bearings are vital components in rotating machinery and play a central role in industrial equipment reliability. Over time, mechanical wear, vibration, and stress contribute to the degradation of bearings. If not detected early, bearing failures can lead to catastrophic equipment damage, production downtime, and increased maintenance costs.

The ability to estimate the **Remaining Useful Life (RUL)** of a bearing is therefore critical. RUL prediction involves estimating the time remaining before a component reaches a failure state, enabling preventive maintenance strategies and optimal resource utilization.

This technical document outlines a comprehensive machine learning-based approach for bearing degradation analysis using real-world vibration data. It includes signal preprocessing, feature engineering, stage-wise labeling of degradation, model training using Support Vector Machines (SVM), and final RUL prediction.

2. Dataset Overview

The dataset used originates from vibration sensors mounted on three bearings operating under various conditions. Each sample consists of time-series data for horizontal and vertical accelerations. The dataset is structured into folders like '$Bearing1-1$', '$Bearing1-2$', etc., each containing thousands of '.csv' files named sequentially.

Key parameters extracted include:

- **RRMS (Relative Root Mean Square)** – A normalized measure of vibration magnitude.
- **IRRMS (Instantaneous RRMS)** – A filtered, trend-tracking version of RRMS that smooths sudden spikes.
- **Degradation Labels** – Class labels (0 to 4) indicating the bearing's condition.

3. Methodology

3.1. Data Loading and Preprocessing

CSV files are loaded using 'pandas', and the horizontal acceleration data (column 5) and vertical acceleration data (column 6) are extracted. For each file, RMS is calculated, and missing or malformed files are skipped.

3.2. Feature Extraction: RRMS and IRRMS

The RRMS is computed to represent vibration energy. It is normalized against the first 250 samples to ensure consistency across different bearings. The IRRMS is then calculated using a moving window and linear regression to identify trend slopes and suppress local noise.

Code Snippet (RRMS Calculation):

```
def rrms(rms):
    stable_data_point = 250
    rms_np = np.array(rms)
    rms_norm = np.mean(rms_np[:stable_data_point])
    return rms_np / rms_norm
```

3.3. IRRMS Estimation with Outlier Detection and Trend Filtering

To better capture long-term trends in the RRMS signal and suppress short-term fluctuations, we define an additional feature: the **Instantaneous RRMS (IRRMS)**. This signal reflects the underlying degradation trend more reliably than raw RRMS.

Mathematical Approach

The IRRMS at a given time step is computed using a local linear fit over a moving window of size w (e.g., 30 samples). For each window position i, we define:

- $x_i = 10i$: the time at the i-th sample (assuming 10s interval).
- y_i: the mean of RRMS values within the window $[i, i + w)$.

We then compute the best-fit line using simple linear regression:

$$k = \frac{\sum(x_j - \bar{x})(y_j - \bar{y})}{\sum(x_j - \bar{x})^2}, \quad b = \bar{y} - k\bar{x}$$

Where:

- k is the slope of the local trend
- b is the intercept
- $\bar{x}, \bar{y}$ are the means of the time and signal values within the window

Outlier Detection Using Residual Analysis

We calculate the deviation (residual) from the fitted trend for each point in the window:

$$d_j = \text{RRMS}_j - (kx_j + b)$$

Let μ and σ be the mean and standard deviation of the residuals d_j. We define acceptable bounds using the 3σ rule:

$$\text{Bounds: } [\mu - 3\sigma, \mu + 3\sigma]$$

Now, the current point's deviation d_i is tested against these bounds:

- If $d_i < \mu - 3\sigma$: assign IRRMS as the local window average (i.e., smooth flat region).
- If $d_i > \mu + 3\sigma$: assign IRRMS as $kx_i + b + 3\sigma$ (preserves rising trend but clips spike).
- Else: assign IRRMS as $kx_i + b$ (fitted line).

Final IRRMS Calculation Logic

```
if rrms_d_i <= min_rrms_d:
    temp_irrms = np.mean(rrms_np[i: i+window_length])
elif rrms_d_i >= max_rrms_d:
    temp_irrms = k * x[i] + b + max_rrms_d
else:
    temp_irrms = k * x[i] + b
```

This procedure ensures IRRMS captures long-term degradation trends while robustly handling noise and measurement spikes.

Benefits

- Smooths the input signal for improved learning stability.
- Enhances model sensitivity to actual degradation trends.
- Provides a slope value (k) for further use in life progression estimation.

3.4. Degradation Stage Label Assignment

After the computation of RRMS and IRRMS features, the next step is to classify the timeline of bearing life into distinct stages of degradation. This classification provides the supervised labels required to train the SVM model.

Degradation Onset Detection

To determine the onset of degradation, a predefined threshold θ_{rrms} (typically 1.1) is used. This threshold marks the deviation from the normal operating state of the bearing. The RRMS vector is scanned in reverse to identify the last point where:

$$\text{RRMS}_i < \theta_{\text{rrms}}$$

Let:

- N be the total number of RRMS samples.
- deg_rev_idx be the index from the end where RRMS dips below the threshold.
- deg_idx $= N -$ deg_rev_idx be the degradation onset index.
- deg_time $=$ deg_idx $\times$ 10 seconds (assuming 10-second sampling).

All data prior to deg_idx is considered healthy (Stage 0).

Stage Segmentation of Failure Region

The failure period after deg_idx is segmented into four progression stages. The division is based on percentages of the failure window length deg_rev_idx:

- **Stage 1 (Initial degradation):** First 60% of degradation window
- **Stage 2 (Moderate degradation):** Next 20%
- **Stage 3 (Severe degradation):** Next 15%
- **Stage 4 (Near-failure):** Final 5%

This segmentation reflects increasing severity of bearing wear as the failure point approaches.

Mathematical Mapping of Labels

The label vector Y is assigned based on the sample index i relative to deg_idx and deg_rev_idx:

$$Y_i = \begin{cases} 0, & \text{if } i < \text{deg_idx} \\ 1, & \text{if deg_idx} \leq i < \text{deg_idx} + 0.6 \times \text{deg_rev_idx} \\ 2, & \text{if } 0.6 \times \text{deg_rev_idx} \leq i < 0.8 \times \text{deg_rev_idx} \\ 3, & \text{if } 0.8 \times \text{deg_rev_idx} \leq i < 0.95 \times \text{deg_rev_idx} \\ 4, & \text{otherwise} \end{cases}$$

Code Snippet (Label Assignment Logic)

```
for i in range(deg_idx):
    output_data.append(0)  # Stage 0

for i in range(deg_idx, len(rrms)):
    if i < deg_idx + int(deg_rev_idx * 0.6):
        output_data.append(1)
    elif i < deg_idx + int(deg_rev_idx * 0.8):
        output_data.append(2)
    elif i < deg_idx + int(deg_rev_idx * 0.95):
        output_data.append(3)
    else:
        output_data.append(4)
```

Rationale

This stage-based approach allows the machine learning model to capture nonlinear degradation patterns and provides finer granularity than a binary healthy/failure classification. It also aligns with real-world scenarios where machine health deteriorates progressively, not instantaneously.

3.5. 6. Model Training: Support Vector Machine (SVM)

After computing the feature vectors (RRMS and IRRMS) and assigning degradation stage labels, a classification model is trained to learn the relationship between feature patterns and corresponding degradation stages. For this purpose, a Support Vector Classifier (SVC) with a Radial Basis Function (RBF) kernel is employed due to its effectiveness in handling non-linear decision boundaries and high-dimensional feature spaces.

6.1. Feature Vector Construction

Each data point consists of two key features:

- **RRMS**: Relative Root Mean Square – normalized vibration energy signal.
- **IRRMS**: Iteratively Refined RRMS – smoothed trend line with outlier removal.

These features are concatenated for each time step to form a 2D feature vector:

$$\mathbf{x}_i = [\mathrm{RRMS}_i, \mathrm{IRRMS}_i] \quad \text{for } i = 1, \ldots, N$$

6.2. Data Preparation and Normalization

The training dataset comprises all bearings' data concatenated together:

$$\mathbf{X}_{\text{train}} = \mathbf{X}_{1_1} \cup \mathbf{X}_{1_2} \cup \cdots \cup \mathbf{X}_{3_2} \quad \mathbf{Y}_{\text{train}} = \mathbf{Y}_{1_1} \cup \mathbf{Y}_{1_2} \cup \cdots \cup \mathbf{Y}_{3_2}$$

To ensure uniform feature scaling and accelerate model convergence, a Min-Max normalization is applied:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

This transformation scales both RRMS and IRRMS features to the $[0, 1]$ interval.

6.3. Model Training Using SVM

The model is implemented using the 'SVC()' class from `scikit-learn` with the following configuration:

- **Kernel:** Radial Basis Function (RBF) – suitable for non-linear boundaries
- **C:** Regularization parameter set to 10 to allow some misclassification while maximizing the margin
- **Gamma:** Set to `scale`, allowing the kernel to adapt to the dataset

$$\text{model} = \texttt{SVC(kernel='rbf', C=10, gamma='scale')}$$

The model is trained on the scaled input vectors and their corresponding labels:

$$\text{model.fit}(\mathbf{X}_{\text{train}}, \mathbf{Y}_{\text{train}})$$

6.4. Model Persistence

To support deployment in real-time systems, the trained classifier and the scaler are serialized using `joblib`:

```
dump(model, 'svm_classifier_app.pkl')
dump(scaler, 'svm_scaler_app.pkl')
```

6.5. Justification for SVM

SVM is chosen due to the following advantages in industrial machine health classification:

- High accuracy and robustness in binary and multiclass classification problems
- Effective for high-dimensional datasets with small sample sizes
- Handles non-linear class separation through the RBF kernel

6.6. Output of Training

The final model is capable of predicting one of five degradation stages (0–4) from RRMS and IRRMS signals. These predictions are subsequently used in the RUL estimation pipeline.

4. Life Progression Estimation

4.1. Overview

To estimate the Remaining Useful Life (RUL) of a bearing at any point in time, we map each predicted degradation stage to a corresponding percentage of the component's life. This percentage, denoted as `lp`, ranges from 0 (completely healthy) to 1.0 (end-of-life). The mapping is used to scale degradation stages to time-domain RUL predictions.

The vector `lp` is initialized as:

$$\texttt{lp} = [0, 0.60, 0.80, 0.95, 1.0]$$

These values represent the estimated life progression associated with each degradation stage (0–4). However, to account for differences in degradation rates across bearings and time windows, `lp` is dynamically updated based on signal behavior.

4.2. Local Slope Estimation

The degradation rate is estimated from the slope of the IRRMS signal over a sliding window:

$$k = \frac{n \sum (x_i y_i) - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$$

Where:

- x_i: Time indices within the window
- y_i: Corresponding average IRRMS values
- k: Linear slope representing degradation rate

Interpretation:

- **Small** $k \Rightarrow$ Slow or stable progression — minor `lp` update
- **Large** $k \Rightarrow$ Rapid degradation — aggressive `lp` update

4.3. Hybrid Degradation Logic

The function `hybrid_degradation()` updates `lp` based on the calculated slope k as follows:

- If $k \leq k_0$ (default: 0.0005), degradation is considered slow. The progression is averaged with the previous stage:

$$\texttt{lp[stage]} \leftarrow \frac{\texttt{lp[stage]} + \texttt{lp[stage - 1]}}{2}$$

- If $k > k_0$, degradation is accelerating. The update is scaled using the difference between consecutive IRRMS values and normalized over that stage's IRRMS bounds:

$$\texttt{lp[stage]} \leftarrow \texttt{lp[stage - 1]} + \frac{(\texttt{irrms}_j - \texttt{irrms}_{j-1}) \cdot (\texttt{lp[stage]} - \texttt{lp[stage - 1]})}{\texttt{irrms_edges}[stage][1] - \texttt{irrms_edges}[stage][0]}$$

4.4. Code Support (Excerpt)

```
if k <= k0:
    lp[stage] = (lp[stage] + lp[stage - 1]) / 2
else:
    lp[stage] = (irrmsj - irrmsj1) * (lp[stage] - lp[stage - 1])
    lp[stage] = lp[stage] / (irrms_edges[stage][1] - irrms_edges[
       stage][0])
    lp[stage] = lp[stage] + lp[stage - 1]
```

5. Remaining Useful Life (RUL) Prediction

5.1. Overview

In our approach, RUL is computed based on:

- The current degradation stage (predicted by the trained SVM classifier)
- The precomputed degradation onset time (`deg_time`)
- The life progression mapping (`lp`) for the current stage

5.2. Mathematical Formula

Let:

- t_i = current time index (in seconds)
- `deg_time` = timestamp where degradation begins (i.e., RRMS drops below threshold)
- `lp[class]` = life progression value for the predicted class

Therefore the predicted Remaining Useful Life (RUL) at time t_i is:

$$RUL_i = \texttt{Failure_Time} - t_i = \left(\texttt{deg_time} + \frac{(t_i - \texttt{deg_time})}{\texttt{lp[class]}}\right) - t_i$$

5.3. Interpretation

- When the model predicts `class` = 0 (healthy), RUL is not calculated — assumed to be stable.

- For `class` > 0, RUL decreases dynamically as t_i increases.
- The equation accounts for varying degradation speeds by scaling the remaining life with the stage-specific life progression.

5.4. Code Support (Excerpt)

```
if val > 0:
    rul_pred = deg_time + ((idx * 10) - deg_time) / lp[val] - (idx *
        10)
    rul_pred_total = deg_time + ((idx * 10) - deg_time) / lp[val]
else:
    rul_pred = 0
```

6. Fault Condition Classification

6.1. Post-RUL Thresholding for Decision Making

Once the Remaining Useful Life (RUL) has been computed for a bearing, a threshold-based condition classification is performed to determine whether immediate maintenance intervention is necessary.

6.2. Logic Description

The threshold is based on a critical RUL value of 30 minutes (i.e., 1800 seconds). If the predicted RUL falls below this value, the system flags the bearing as being close to failure, specifically indicating the likelihood of an **inner race fault**. Otherwise, the bearing is considered to be operating in a **normal/healthy** condition.

Decision Rule:

$$\text{Condition} = \begin{cases} \texttt{"Inner Race Fault Incoming"} & \text{if } RUL_i < 1800 \text{ seconds} \\ \texttt{"Normal / Healthy"} & \text{otherwise} \end{cases}$$

6.3. Code Snippet (Condition Classification)

```
condition_status = []
for rul in rul_pred:
    if rul < 1800:
        condition_status.append("Inner␣Race␣Fault␣Incoming")
```

```
    else:
        condition_status.append("Normal␣/␣Healthy")
```

7. Conclusion

This methodology provides an end-to-end pipeline from raw vibration data to actionable predictions on bearing health. By extracting trend features (RRMS, IRRMS), assigning degradation stages, and training a robust classifier, the system enables industrial equipment operators to schedule maintenance intelligently, reduce unplanned downtime, and ensure operational efficiency.